

Different Types of Memory in LLMs

When people talk about **memory in LLM applications**, they can mean several different things. A useful way to think about it is from shortest-lived to longest-lived.

1. Context Window Memory (Short-Term Memory)

This is the memory built into the model during a conversation.

When you chat:

You: My name is Sudip.

You: What's my name?

The model remembers because both messages are still inside the current context window.

System Prompt
+
Conversation History
+
Current Question

Everything inside this window is the model's "working memory."

Limitation

If the conversation becomes too long:

Message 1
Message 2
...
Message 500

older messages may be dropped.

Then the model forgets them.

2. Buffer Memory (Conversation History)

Applications often explicitly store previous messages.

Example:

```
history = [  
    HumanMessage("My name is Sudip"),  
    AIMessage("Nice to meet you"),  
]
```

Each new request sends:

```
Previous Messages  
+  
Current Question
```

to the LLM.

Advantage

Easy to implement.

Problem

History keeps growing and becomes expensive.

3. Conversation Summary Memory

Instead of storing every message:

```
User likes Java.  
User studies at NIT Jamshedpur.  
User is preparing for software interviews.
```

the application periodically creates a summary.

Example:

```
Summary:  
Sudip is an MCA student at NIT Jamshedpur preparing for software engineering  
roles.
```

Then only the summary is sent.

Advantage

Saves tokens.

Disadvantage

Can lose details.

4. Vector Memory (Semantic Memory)

Store information as embeddings in a vector database.

Suppose earlier you said:

I prefer Java for DSA.

This gets embedded and stored.

Later:

Which language should I use for coding interviews?

The system retrieves the relevant memory and injects it into the prompt.

Flow:

```
Conversation
  |
Embedding
  |
Vector Store
  |
Similarity Search
  |
Relevant Memories
```

Examples:

- Chroma
 - FAISS
 - Pinecone
-

5. Entity Memory

Stores facts about specific entities.

Example:

```
User:  
My name is Sudip.  
  
I study at NIT Jamshedpur.  
  
I like Java.
```

The memory system extracts:

```
{  
  "name": "Sudip",  
  "college": "NIT Jamshedpur",  
  "favorite_language": "Java"  
}
```

Then later:

```
What language should I practice?
```

The assistant can use:

```
favorite_language = Java
```

without searching the entire chat history.

6. Long-Term Memory (Persistent Memory)

Memory survives across sessions.

Today:

```
I am preparing for SDE interviews.
```

Next week:

How should I continue my preparation?

The system can still remember.

This requires external storage:

Database
Vector Store
Profile Store
Knowledge Graph

rather than relying on the model itself.

7. Episodic Memory

Inspired by human memory.

Stores experiences/events.

Example:

On June 1:
Solved 5 graph problems.

On June 5:
Started learning LangGraph.

Memory becomes:

Episode 1
Episode 2
Episode 3

Agents can retrieve relevant past experiences.

8. Semantic Memory

Stores knowledge and facts.

Example:

```
User prefers Java.  
User is interested in AI agents.  
User has solved 500+ LeetCode problems.
```

These are facts, not events.

Human analogy:

```
Episodic = I remember visiting Delhi.  
Semantic = I know Delhi is India's capital.
```

9. Working Memory (Agent Memory)

Used during a single task.

Suppose an agent is solving:

```
Find hotels  
Compare prices  
Book hotel  
Send confirmation
```

It keeps temporary state:

```
{  
  "city": "Delhi",  
  "budget": 5000,  
  "selected_hotel": "XYZ"  
}
```

This memory disappears when the task ends.

In LangGraph, this is often the graph state.

Memory in LangChain

Historically:

```
ConversationBufferMemory  
ConversationSummaryMemory  
ConversationBufferWindowMemory
```

```
VectorStoreRetrieverMemory  
EntityMemory
```

These classes existed to manage different memory styles.

Modern LangGraph applications often replace these with:

```
State  
+  
Checkpointter  
+  
Vector Database  
+  
Persistent Storage
```

which gives more control.

Memory in LangGraph

A LangGraph agent typically uses:

Short-Term Memory

Graph state:

```
state["messages"]
```

Long-Term Memory

External storage:

```
Postgres  
Redis  
Vector DB  
Knowledge Graph
```

Checkpoint Memory

Stores state between graph executions.

Examples:

- MemorySaver

- Postgres checkpoint

Human Analogy

Human Memory	LLM Equivalent
Working memory	Context window
Short-term memory	Conversation history
Notes notebook	Vector database
Personal profile	Entity memory
Life experiences	Episodic memory
General knowledge	Semantic memory
Long-term memory	Persistent storage

Interview Answer

Memory in LLM systems can be categorized into short-term memory (context window and conversation history), summarized memory (conversation summaries), semantic memory (facts stored in vector databases), episodic memory (past events and experiences), entity memory (structured user facts), and long-term persistent memory stored externally. Modern agent frameworks such as LangGraph typically combine graph state for working memory with databases, vector stores, and checkpointing systems for long-term memory.